

小组报告 — TurtleMaze

基于 ROS2 的迷宫探索与手机遥控系统

项目名称: TurtleMaze
课程: Week14 小组项目
日期: 2026-06-29
ROS 版本: ROS2 Humble

一、项目概述与目标

TurtleMaze 是一个基于 ROS2 Humble 的 Turtlesim 迷宫探索系统，核心目标是将经典的 Turtlesim 仿真器扩展为一个完整的迷宫探索平台。

项目包含两大核心能力：

- 自动迷宫探索算法**：基于深度优先搜索（DFS）策略，驱动小乌龟自动遍历迷宫，实现壁障检测与路径回溯。
- 基于 WebSocket 的手机遥控系统**：通过 Web 网页实现对乌龟的实时遥控，支持虚拟摇杆和速度调节。

项目采用 ROS2 ament_python 包结构组织，支持一键 Launch 启动，同时提供无需编译的脚本运行模式，降低使用门槛。

二、技术架构

系统采用分层架构设计，自下而上分为五层：

层级	名称	核心技术	说明
L1	机器人仿真层	ROS2 Turtlesim 仿真器	提供乌龟运动、画笔轨迹、碰撞检测
L2	通信中间件层	ROS2 Topic/Service	<code>/turtle1/cmd_vel</code> 、 <code>/turtle1/pose</code> 等话题通信
L3	控制算法层	explorer.py (DFS)	壁障检测 + 栈结构路径回溯
L4	桥接服务层	turtlesim_web_bridge.py (aiohttp + WebSocket)	桥接 ROS2 与 Web 前端，双向数据转发
L5	用户交互层	index.html (虚拟摇杆)	手机端遥控界面，响应式设计

数据流说明：

- 下行情境（遥控 → 乌龟）**：手机网页 → WebSocket → bridge → ROS2 `/turtle1/cmd_vel` Topic → Turtlesim
- 上行情境（位姿 → 手机）**：Turtlesim → ROS2 `/turtle1/pose` Topic → bridge → WebSocket → 手机网页
- 自动探索情境**：explorer 订阅 `/turtle1/pose`，计算下一步移动方向，发布至 `/turtle1/cmd_vel`

三、各模块详细功能描述

3.1 turtlesim_web_bridge.py — WebSocket 桥接主程序

负责 ROS2 与 Web 前端之间的双向通信桥接：

- 使用 aiohttp 创建 WebSocket 服务器，监听 `0.0.0.0:8765`
- 创建 ROS2 节点，订阅 `/turtle1/pose` 话题获取乌龟实时位姿
- 将从 Web 前端接收的控制指令（线速度 / 角速度）封装为 `geometry_msgs/Twist` 消息，发布至 `/turtle1/cmd_vel`
- 将乌龟实时位姿通过 WebSocket 推送给前端，实现状态同步
- 支持两种运行模式：脚本模式（直接 `python3` 运行）和 ROS2 Launch 模式

3.2 explorer.py — 自动迷宫探索算法

实现基于 DFS（深度优先搜索）的迷宫自动遍历：

- 订阅 `/turtle1/pose` 获取当前位置，发布 `/turtle1/cmd_vel` 控制运动
- 壁障检测：利用乌龟位姿与迷宫边界判断，遇到墙壁则标记并转向
- 路径回溯：使用栈结构维护探索路径，遇到死路自动回溯到上一分叉点继续探索未访问方向
- 支持迷宫完整遍历和路径记录

3.3 index.html — 手机遥控网页

纯前端单页面应用，无需任何构建工具：

- 实现虚拟摇杆（Touch Joystick），支持触摸拖拽控制乌龟运动方向
- 速度滑块：支持独立调节线速度和角速度
- 通过 WebSocket 连接后端 bridge 服务，实现低延迟实时控制
- 实时显示乌龟当前坐标和朝向角度
- 响应式设计，针对手机屏幕尺寸优化

3.4 turtle_maze.launch.py — ROS2 Launch 文件

一键启动所有系统组件：

- 启动 turtlesim 仿真器节点
- 启动 WebSocket bridge 节点
- 启动 explorer 探索节点（可通过参数控制是否启用）
- 配置节点参数（迷宫尺寸、运动速度等）

3.5 包管理配置文件

- package.xml**：声明 ROS2 包依赖，包括 `rcpp`、`geometry_msgs`、`turtlesim`、`aiohttp`
- setup.py**：定义三个 `entry_points` 可执行节点（bridge、explorer、launch）
- setup.cfg**：Python 包元信息配置

四、运行方式与使用说明

方式一：ROS2 Launch（推荐）

```
# 1. 编译包
colcon build --packages-select turtle_maze

# 2. 加载环境
source install/setup.bash

# 3. 一键启动
ros2 launch turtle_maze turtle_maze.launch.py
```

方式二：脚本模式（无需编译）

```
# 1. 安装 Python 依赖
pip install -r requirements.txt

# 2. 启动 Turtlesim 仿真器
ros2 run turtlesim turtlesim_node

# 3. 启动 WebSocket Bridge（新终端）
python3 turtlesim_web_bridge.py

# 4. 启动自动探索（新终端，可选）
python3 explorer.py
```

手机遥控

- 确保手机与运行 ROS2 的电脑处于同一局域网
- 手机浏览器访问 `http://<电脑IP地址>:8765`
- 使用虚拟摇杆操控乌龟移动，通过滑块调节速度

五、小组成员与分工

序号	姓名	学号	负责模块	备注
1	(待填写)	(待填写)	(待填写)	
2	(待填写)	(待填写)	(待填写)	
3	(待填写)	(待填写)	(待填写)	
4	(待填写)	(待填写)	(待填写)	

六、总结

TurtleMaze 项目成功将 ROS2 Turtlesim 仿真器扩展为具备自动探索与远程遥控能力的迷宫平台。项目采用分层架构设计，通过 ROS2 Topic 通信实现仿真器与算法层的数据交换，利用 WebSocket 桥接技术打通 Web 前端与 ROS2 的实时通信链路。

项目在工程实践上兼顾**规范性**与**易用性**：一方面严格遵循 ROS2 ament_python 包标准，支持 colcon 编译和 Launch 一键启动；另一方面提供脚本运行模式，允许开发者在无需完整编译流程的情况下快速调试。

系统的模块化设计使得各组件（bridge、explorer、web 前端）均可独立运行和测试，降低了调试复杂度，也为后续功能扩展（如多乌龟协同、A* 算法替换 DFS、迷宫自动生成等）预留了清晰的扩展接口。

